BDY1103 | EP2

Informe de actualización

Proyecto PL/SQL Gas al Paso

Continuidad del proyecto iniciado el 18 de septiembre de 2026

Integrantes

Gerard Zambrano · Matias Echeverria · Jorge Zarate

Fecha de creación inicial: 18/09/2026

Revisión: 1.1 | Fecha de revisión: 27/10/2026

Control de revisión y continuidad

Este documento actualiza el informe de la primera etapa. Mantiene como antecedente el trabajo iniciado el 18/09/2026 y describe el avance vigente con el script SQL y las evidencias de ejecución recibidas.

Versión	Fecha	Descripción
Inicial	18/09/2026	Base del proyecto: inventario, reportes de ventas, validaciones de cantidad y primeros objetos PL/SQL.
Revisión 1.1	27/10/2026	Registro de ventas por empleado y sucursal, detalle de productos, control de stock, resumen diario y auditoría persistente de precios.

Qué se conserva de la etapa anterior

La primera versión trabajó reportes de inventario y ventas por tienda, colecciones de productos, cursores y reglas de validación. Ese trabajo sigue siendo el punto de partida del proyecto. En esta revisión, la exposición se centra en registrar ventas, comprobar existencias y consultar los resultados por empleado y por día.

Qué cambia en esta revisión

- El ejemplo actual usa las tablas **Empleado**, **Venta** y **Detalle_venta**, junto con Tienda y Productos.
- El cálculo vigente aplica un 10% de descuento cuando el subtotal alcanza $30.000. El informe anterior describía otra regla para ventas por cantidad; aquí se documenta la regla que aparece en el SQL actual.
- El package actual se llama **PkgVentas**. La revisión anterior mencionaba UpdateStock e InventarioPackage.
- El cambio de precio queda guardado en **Auditoria_Precio**; ya no depende solo de un mensaje en pantalla.

La fecha 18/09/2026 identifica el inicio de la versión anterior. La fecha 27/10/2026 corresponde a esta revisión del informe y del proyecto.

1. Proyecto, alcance y modelo actual

Gas al Paso vende balones de gas y reguladores. La actualización busca dejar cada venta asociada a un producto, un empleado y una sucursal, aplicar el descuento que corresponda y facilitar la revisión de existencias y ventas del día.

Alcance de esta versión

El script crea una base de ejemplo con dos sucursales, dos empleados y cinco productos. Cada llamada a RegistrarVenta registra un producto y una cantidad. El modelo permite relacionar las ventas con la sucursal y el empleado, y conserva el precio aplicado en el detalle. El stock se maneja como cantidad general del producto, no separado por tienda.

Tecnologías usadas

Oracle Database y SQL Developer para ejecutar las sentencias; PL/SQL para funciones, procedimientos, package y triggers; secuencias para generar folios; y DBMS_OUTPUT para mostrar resultados y mensajes de prueba.

Tablas y relaciones



Relaciones principales: Tienda-Empleado, Tienda-Venta, Empleado-Venta, Venta-Detalle_venta y Productos-Detalle_venta/Auditoria_Precio.

Venta guarda la fecha y los importes totales. Detalle_venta conserva el producto, cantidad y precio de cada línea. Auditoria_Precio conserva el precio anterior, el nuevo y la fecha del cambio.

2. Caso 1: calcular subtotal y descuento

Las funciones almacenadas reciben datos y devuelven un resultado. En este caso, CalcularSubtotal busca el precio vigente del producto y lo multiplica por la cantidad. También rechaza cantidades inválidas y productos inexistentes. CalcularDescuento devuelve el 10% del subtotal cuando este es igual o mayor a $30.000; bajo ese monto devuelve cero.

Fragmento de CalcularDescuento

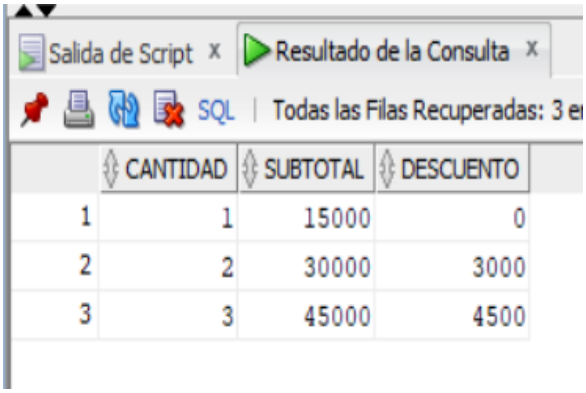
```
CREATE OR REPLACE FUNCTION CalcularDescuento (
    p_subtotal IN NUMBER
) RETURN NUMBER IS
BEGIN
    IF p_subtotal >= 30000 THEN
        RETURN ROUND(p_subtotal * 0.10, 2);
    END IF;
    RETURN 0;
END CalcularDescuento;
```

Uso en el negocio

Las funciones se pueden llamar desde una consulta SQL o desde otro bloque PL/SQL. RegistrarVenta usa CalcularDescuento para completar subtotal, descuento y total antes de guardar la venta.

Ejemplo: dos balones de 11 kg a $15.000 suman $30.000. El descuento es $3.000 y el cliente paga $27.000.

Ventaja: el cálculo queda en un solo lugar y se reutiliza. Cuidado: si una función hace muchas consultas o tareas, puede demorar los procesos que la llaman.



	CANTIDAD	SUBTOTAL	DESCUENTO
1	1	15000	0
2	2	30000	3000
3	3	45000	4500

Resultado de la consulta con cantidades 1, 2 y 3. Para $30.000 el descuento es $3.000; para $45.000, es $4.500.

3. Caso 2: registrar una venta con PkgVentas

Un package (paquete) reúne rutinas que trabajan juntas. La especificación de PkgVentas deja disponible RegistrarVenta. El cuerpo del package contiene ValidarDatos, que se usa internamente para revisar cantidad, empleado y producto, y obtener la sucursal y el precio.

La rutina pública recibe el empleado, el producto y la cantidad. Devuelve el folio de la venta mediante un parámetro de salida. Luego calcula importes, inserta la cabecera en Venta y el detalle en Detalle_venta. Antes de terminar, el trigger de stock comprueba y descuenta las existencias. Si ocurre un error, el procedimiento vuelve al punto de guardado de la transacción.

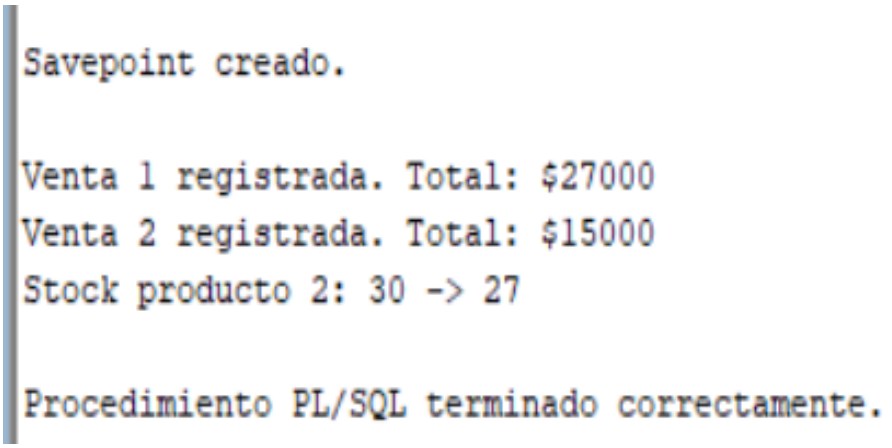
Especificación pública

```
CREATE OR REPLACE PACKAGE PkgVentas IS
    PROCEDURE RegistrarVenta (
        p_id_empleado IN NUMBER,
        p_id_producto IN NUMBER,
        p_cantidad IN NUMBER,
        p_id_venta OUT NUMBER
    );
END PkgVentas;
```

Relación con funciones y procedimientos

RegistrarVenta llama a CalcularDescuento. MostrarBoleta y los procedimientos de resumen se ejecutan por separado para consultar lo que quedó registrado. La separación ayuda a mantener cada tarea clara. La desventaja es que el cuerpo del package puede volverse difícil de mantener si acumula demasiadas responsabilidades.

Evidencia de registro



Dos ventas de ejemplo: $27.000 y $15.000. El stock del producto 2 pasa de 30 a 27 unidades.

La prueba final confirma las ventas, empleados, sucursales, totales y stock.

4. Caso 2: triggers para cuidar los datos

Un trigger es una acción que Oracle ejecuta automáticamente cuando una tabla recibe una operación. El proyecto usa triggers para revisar stock, proteger ventas confirmadas y registrar cambios de precio.

Controles de fila y de sentencia

- **TrgDetalleStock:** se ejecuta antes de insertar cada detalle. Revisa el stock disponible y detiene la venta si no alcanza; si alcanza, descuenta la cantidad.
- **TrgVentaConfirmada** y **TrgDetalleConfirmado:** bloquean actualizaciones y eliminaciones de ventas y detalles confirmados. Se ejecutan por sentencia.
- **AuditoriaCambioPrecio:** se ejecuta por fila después de cambiar el precio de un producto. Guarda el valor anterior, el nuevo y la fecha en Auditoria_Precio.

Ejemplo del control de stock

```
IF v_stock < :NEW.cantidad THEN
    RAISE_APPLICATION_ERROR(-20003,
        'Stock insuficiente. Disponible: ' || v_stock);
END IF;

UPDATE Productos
SET stock = stock - :NEW.cantidad
WHERE id_producto = :NEW.id_producto;
```

Los triggers mantienen reglas importantes aunque la venta se registre desde otro bloque. Como se activan automáticamente, sus efectos pueden ser menos evidentes al buscar un error; por eso conviene mantenerlos breves y probar los casos de rechazo.

Evidencia de auditoría y validación

```
Procedimiento PL/SQL terminado correctamente.

1 fila actualizadas.


ID_PRODUCTO PRECIO_ANTERIOR PRECIO_NUEVO
----------- --------------- ------------
          1            8000         8500

Rollback terminado.
```

```
Procedimiento PL/SQL terminado correctamente.
ORA-20005: No existe el empleado o producto indicado

Procedimiento PL/SQL terminado correctamente.
ORA-20003: Stock insuficiente. Disponible: 5

Procedimiento PL/SQL terminado correctamente.
```

Precio de $8.000 a $8.500 guardado en auditoría. Las pruebas también detienen empleado inexistente y stock insuficiente.

5. Caso 3: boleta y resumen de ventas

Los procedimientos realizan una acción; pueden recibir parámetros y mostrar información. Las funciones, en cambio, devuelven un valor que se puede usar en otra operación.

Objetos disponibles

Objeto	Uso en el proyecto	Parámetro
MostrarBoleta	Muestra folio, fecha, empleado, sucursal, productos e importes.	Recibe el folio de venta.
ResumenEmpleado	Cuenta las ventas y suma lo vendido por una persona durante el día.	Recibe el identificador del empleado.
ResumenHoy	Recorre las ventas del día y muestra cantidad e importe total.	No recibe parámetros.
TotalVendidoHoy	Devuelve el importe total del día; también se usa como control del resumen.	No recibe parámetros; devuelve un número.

Resultados del resumen diario

La captura muestra dos ventas, por $27.000 y $15.000. El total calculado por ResumenHoy coincide con el control de TotalVendidoHoy: $42.000.

```
Procedimiento PL/SQL terminado correctamente.
----- RESUMEN DEL DIA 25-10-2026 -----
Venta 1: $27000
Venta 2: $15000
Ventas de hoy: 2
Total vendido: $42000
Control: $42000

Procedimiento PL/SQL terminado correctamente.
```

Salida de prueba del 25/10/2026.

La boleta es una salida académica de DBMS_OUTPUT. Sirve para revisar la operación, pero no reemplaza una boleta tributaria electrónica.

6. Pruebas, impacto y límites actuales

Las pruebas usan datos de ejemplo para comprobar que una venta correcta quede vinculada a su empleado y sucursal, que el total sea correcto y que el stock baje. También se prueban situaciones que deben rechazarse.

Prueba	Resultado observado
Dos balones de 11 kg	Subtotal $30.000, descuento $3.000, total $27.000.
Dos ventas de empleados distintos	Totales de $27.000 y $15.000; el stock baja de 30 a 27.
Resumen del día	Dos ventas; total y control coinciden en $42.000.
Empleado inexistente o stock insuficiente	Oracle detiene el registro y muestra el motivo.
Cambio de precio	Auditoria_Precio conserva precio anterior y precio nuevo.

Impacto para Gas al Paso

La actualización reduce cuentas manuales y permite revisar quién hizo cada venta, en qué sucursal, cuánto se cobró y qué stock queda. El registro de precios ayuda a conocer cuándo cambió el valor de un producto.

Límites y mejoras siguientes

- Registrar varios productos dentro de una misma venta.
- Separar las existencias por sucursal, en vez de manejar un único stock general.
- Agregar medios de pago, cierre de caja y comparación entre el dinero esperado y el recibido.
- Conservar pruebas de actualización y rechazo en cada cambio futuro del script.

7. Conclusión

Esta revisión continúa el trabajo iniciado el 18/09/2026 y actualiza su foco. La versión vigente calcula los importes, registra ventas con empleado y sucursal, descuenta existencias, entrega boletas y resúmenes, y mantiene un historial de precios.

Las funciones concentran los cálculos; PkgVentas organiza el registro; los triggers hacen cumplir controles; y los procedimientos muestran información para revisar el día. En conjunto, los resultados comprobados permiten seguir la venta desde el ingreso de los datos hasta el resumen diario.

Recomendación

Como siguiente paso, se recomienda ampliar RegistrarVenta para aceptar varios productos y separar el stock por sucursal. Luego se pueden incorporar pagos y cierre de caja, conservando las validaciones y evidencias de esta revisión.

Versión inicial del proyecto: 18/09/2026 | Informe actualizado: revisión 1.1, 27/10/2026

Anexo A. Código fuente de la versión actual

Se incorpora el script recibido para esta revisión. Se conserva como fuente de referencia de tablas, datos de ejemplo, funciones, package, procedimientos, triggers y pruebas.

Anexo A. Código completo (1/9)

```
SET SERVEROUTPUT ON

-- Preparacion
DECLARE
    PROCEDURE Ejecutar(p_sql VARCHAR2) IS
    BEGIN
        EXECUTE IMMEDIATE p_sql;
    EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE NOT IN (-942, -2289) THEN
                RAISE;
            END IF;
    END;
BEGIN
    Ejecutar('DROP TABLE Detalle_venta CASCADE CONSTRAINTS PURGE');
    Ejecutar('DROP TABLE Auditoria_Precio CASCADE CONSTRAINTS PURGE');
    Ejecutar('DROP TABLE Venta CASCADE CONSTRAINTS PURGE');
    Ejecutar('DROP TABLE Productos CASCADE CONSTRAINTS PURGE');
    Ejecutar('DROP TABLE Empleado CASCADE CONSTRAINTS PURGE');
    Ejecutar('DROP TABLE Tienda CASCADE CONSTRAINTS PURGE');
    Ejecutar('DROP SEQUENCE seq_venta');
    Ejecutar('DROP SEQUENCE seq_detalle_venta');
END;
/

-- Tablas
CREATE TABLE Tienda (
    id_tienda NUMBER PRIMARY KEY,
    nombre_tienda VARCHAR2(50) NOT NULL
);

CREATE TABLE Empleado (
    id_empleado NUMBER PRIMARY KEY,
    nombre VARCHAR2(80) NOT NULL,
    id_tienda NUMBER NOT NULL,
    CONSTRAINT fk_empleado_tienda FOREIGN KEY (id_tienda)
        REFERENCES Tienda(id_tienda)
);

CREATE TABLE Productos (
    id_producto NUMBER PRIMARY KEY,
    nombre VARCHAR2(50) NOT NULL,
    descripcion VARCHAR2(100) NOT NULL,
    peso NUMBER(5,2) NOT NULL,
    valor_unitario NUMBER(10,2) NOT NULL,
    stock NUMBER DEFAULT 0 NOT NULL,
    CONSTRAINT ck_productos_precio CHECK (valor_unitario > 0),
    CONSTRAINT ck_productos_stock CHECK (stock >= 0 AND stock = TRUNC(stock))
);

CREATE TABLE Venta (
    id_venta NUMBER PRIMARY KEY,
    fecha_venta DATE DEFAULT SYSDATE NOT NULL,
    id_empleado NUMBER NOT NULL,
```

Anexo A. Código completo (2/9)

```
    id_tienda NUMBER NOT NULL,
    subtotal NUMBER(12,2) DEFAULT 0 NOT NULL,
    descuento NUMBER(12,2) DEFAULT 0 NOT NULL,
    total_venta NUMBER(12,2) DEFAULT 0 NOT NULL,
    CONSTRAINT fk_venta_empleado FOREIGN KEY (id_empleado)
        REFERENCES Empleado(id_empleado),
    CONSTRAINT fk_venta_tienda FOREIGN KEY (id_tienda)
        REFERENCES Tienda(id_tienda),
    CONSTRAINT ck_venta_importes CHECK (
        subtotal >= 0 AND descuento BETWEEN 0 AND subtotal
        AND total_venta = subtotal - descuento
    )
);

CREATE TABLE Detalle_venta (
    id_detalle NUMBER PRIMARY KEY,
    id_venta NUMBER NOT NULL,
    id_producto NUMBER NOT NULL,
    cantidad NUMBER NOT NULL,
    precio_unitario NUMBER(10,2) NOT NULL,
    subtotal NUMBER(12,2) NOT NULL,
    descuento NUMBER(12,2) NOT NULL,
    total_venta NUMBER(12,2) NOT NULL,
    CONSTRAINT fk_detalle_venta FOREIGN KEY (id_venta)
        REFERENCES Venta(id_venta),
    CONSTRAINT fk_detalle_producto FOREIGN KEY (id_producto)
        REFERENCES Productos(id_producto),
    CONSTRAINT ck_detalle_cantidad CHECK (cantidad > 0 AND cantidad = TRUNC(cantidad)),
    CONSTRAINT ck_detalle_precio CHECK (precio_unitario > 0),
    CONSTRAINT ck_detalle_importes CHECK (
        subtotal = precio_unitario * cantidad
        AND descuento BETWEEN 0 AND subtotal
        AND total_venta = subtotal - descuento
    )
);

CREATE TABLE Auditoria_Precio (
    id_auditoria NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_producto NUMBER NOT NULL,
    precio_anterior NUMBER(10,2) NOT NULL,
    precio_nuevo NUMBER(10,2) NOT NULL,
    fecha_cambio DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT fk_auditoria_producto FOREIGN KEY (id_producto)
        REFERENCES Productos(id_producto)
);

CREATE SEQUENCE seq_venta START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_detalle_venta START WITH 1 INCREMENT BY 1 NOCACHE;

-- Datos de ejemplo
INSERT INTO Tienda VALUES (1, 'Gas al Paso Centro');
INSERT INTO Tienda VALUES (2, 'Gas al Paso Norte');
INSERT INTO Empleado VALUES (1, 'Camila Soto', 1);
INSERT INTO Empleado VALUES (2, 'Diego Rojas', 2);
```

Anexo A. Código completo (3/9)

```
INSERT INTO Productos VALUES (1, 'Gas 5kg', 'Balon de gas licuado 5 kilos', 5, 8000, 50);
INSERT INTO Productos VALUES (2, 'Gas 11kg', 'Balon de gas licuado 11 kilos', 11, 15000, 30);
INSERT INTO Productos VALUES (3, 'Gas 15kg', 'Balon de gas licuado 15 kilos', 15, 20000, 8);
INSERT INTO Productos VALUES (4, 'Gas 45kg', 'Balon de gas licuado 45 kilos', 45, 55000, 5);
INSERT INTO Productos VALUES (5, 'Regulador de gas', 'Regulador universal para balones', 0.3, 4000, 100);

COMMIT;

-- Caso 1: calcular compra y descuento
CREATE OR REPLACE FUNCTION CalcularSubtotal (
    p_id_producto IN NUMBER,
    p_cantidad IN NUMBER
) RETURN NUMBER IS
    v_precio Productos.valor_unitario%TYPE;
BEGIN
    IF p_cantidad IS NULL OR p_cantidad <= 0 OR p_cantidad <> TRUNC(p_cantidad) THEN
        RAISE_APPLICATION_ERROR(-20001, 'La cantidad debe ser un entero mayor que cero');
    END IF;
    SELECT valor_unitario INTO v_precio
    FROM Productos WHERE id_producto = p_id_producto;
    RETURN ROUND(v_precio * p_cantidad, 2);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'El producto no existe');
END CalcularSubtotal;
/

CREATE OR REPLACE FUNCTION CalcularDescuento (
    p_subtotal IN NUMBER
) RETURN NUMBER IS
BEGIN
    IF p_subtotal >= 30000 THEN
        RETURN ROUND(p_subtotal * 0.10, 2);
    END IF;
    RETURN 0;
END CalcularDescuento;
/

-- Caso 2: registrar venta y descontar stock
CREATE OR REPLACE TRIGGER TrgDetalleStock
BEFORE INSERT ON Detalle_venta
FOR EACH ROW
DECLARE
    v_stock Productos.stock%TYPE;
BEGIN
    SELECT stock INTO v_stock
    FROM Productos
    WHERE id_producto = :NEW.id_producto
    FOR UPDATE;
    IF v_stock < :NEW.cantidad THEN
        RAISE_APPLICATION_ERROR(-20003, 'Stock insuficiente. Disponible: ' || v_stock);
    END IF;
    UPDATE Productos
```

Anexo A. Código completo (4/9)

```
    SET stock = stock - :NEW.cantidad
    WHERE id_producto = :NEW.id_producto;
END;
/

CREATE OR REPLACE TRIGGER TrgVentaConfirmada
BEFORE UPDATE OR DELETE ON Venta
BEGIN
    RAISE_APPLICATION_ERROR(-20004, 'No se puede cambiar ni eliminar una venta confirmada');
END;
/

CREATE OR REPLACE TRIGGER TrgDetalleConfirmado
BEFORE UPDATE OR DELETE ON Detalle_venta
BEGIN
    RAISE_APPLICATION_ERROR(-20004, 'No se puede cambiar ni eliminar un detalle confirmado');
END;
/

CREATE OR REPLACE TRIGGER AuditoriaCambioPrecio
AFTER UPDATE OF valor_unitario ON Productos
FOR EACH ROW
BEGIN
    INSERT INTO Auditoria_Precio (id_producto, precio_anterior, precio_nuevo)
    VALUES (:OLD.id_producto, :OLD.valor_unitario, :NEW.valor_unitario);
END;
/

CREATE OR REPLACE PACKAGE PkgVentas IS
    PROCEDURE RegistrarVenta (
        p_id_empleado IN NUMBER,
        p_id_producto IN NUMBER,
        p_cantidad IN NUMBER,
        p_id_venta OUT NUMBER
    );
END PkgVentas;
/

CREATE OR REPLACE PACKAGE BODY PkgVentas IS
    PROCEDURE ValidarDatos (
        p_id_empleado IN NUMBER,
        p_id_producto IN NUMBER,
        p_cantidad IN NUMBER,
        p_id_tienda OUT NUMBER,
        p_precio OUT NUMBER
    ) IS
    BEGIN
        IF p_cantidad IS NULL OR p_cantidad <= 0 OR p_cantidad <> TRUNC(p_cantidad) THEN
            RAISE_APPLICATION_ERROR(-20001, 'La cantidad debe ser un entero mayor que cero');
        END IF;
        SELECT id_tienda INTO p_id_tienda
        FROM Empleado WHERE id_empleado = p_id_empleado;
        SELECT valor_unitario INTO p_precio
        FROM Productos WHERE id_producto = p_id_producto;
```

Anexo A. Código completo (5/9)

```
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20005, 'No existe el empleado o producto indicado');
    END ValidarDatos;

    PROCEDURE RegistrarVenta (
        p_id_empleado IN NUMBER,
        p_id_producto IN NUMBER,
        p_cantidad IN NUMBER,
        p_id_venta OUT NUMBER
    ) IS
        v_id_tienda Empleado.id_tienda%TYPE;
        v_precio Productos.valor_unitario%TYPE;
        v_subtotal NUMBER(12,2);
        v_descuento NUMBER(12,2);
    BEGIN
        SAVEPOINT inicio_venta;
        ValidarDatos(p_id_empleado, p_id_producto, p_cantidad, v_id_tienda, v_precio);
        v_subtotal := ROUND(v_precio * p_cantidad, 2);
        v_descuento := CalcularDescuento(v_subtotal);
        p_id_venta := seq_venta.NEXTVAL;

        INSERT INTO Venta (id_venta, fecha_venta, id_empleado, id_tienda, subtotal, descuento, total_venta)
        VALUES (p_id_venta, SYSDATE, p_id_empleado, v_id_tienda,
                v_subtotal, v_descuento, v_subtotal - v_descuento);

        INSERT INTO Detalle_venta (
            id_detalle, id_venta, id_producto, cantidad,
            precio_unitario, subtotal, descuento, total_venta
        ) VALUES (
            seq_detalle_venta.NEXTVAL, p_id_venta, p_id_producto, p_cantidad,
            v_precio, v_subtotal, v_descuento, v_subtotal - v_descuento
        );

        DBMS_OUTPUT.PUT_LINE('Venta ' || p_id_venta || ' registrada. Total: $' || (v_subtotal - v_descuento));
    EXCEPTION
        WHEN OTHERS THEN
            ROLLBACK TO inicio_venta;
            RAISE;
    END RegistrarVenta;
END PkgVentas;
/

-- Caso 3: boleta y resumen del dia
CREATE OR REPLACE PROCEDURE MostrarBoleta (
    p_id_venta IN NUMBER
) IS
    v_empleado Empleado.nombre%TYPE;
    v_tienda Tienda.nombre_tienda%TYPE;
    v_fecha Venta.fecha_venta%TYPE;
    v_subtotal Venta.subtotal%TYPE;
    v_descuento Venta.descuento%TYPE;
    v_total Venta.total_venta%TYPE;
    v_hay_detalle BOOLEAN := FALSE;
```

Anexo A. Código completo (6/9)

```
BEGIN
    SELECT e.nombre, t.nombre_tienda, v.fecha_venta, v.subtotal, v.descuento, v.total_venta
    INTO v_empleado, v_tienda, v_fecha, v_subtotal, v_descuento, v_total
    FROM Venta v
    JOIN Empleado e ON e.id_empleado = v.id_empleado
    JOIN Tienda t ON t.id_tienda = v.id_tienda
    WHERE v.id_venta = p_id_venta;

    DBMS_OUTPUT.PUT_LINE('----- BOLETA GAS AL PASO -----');
    DBMS_OUTPUT.PUT_LINE('Folio: ' || p_id_venta);
    DBMS_OUTPUT.PUT_LINE('Fecha: ' || TO_CHAR(v_fecha, 'DD-MM-YYYY HH24:MI'));
    DBMS_OUTPUT.PUT_LINE('Empleado: ' || v_empleado);
    DBMS_OUTPUT.PUT_LINE('Sucursal: ' || v_tienda);

    FOR r IN (
        SELECT p.nombre, d.cantidad, d.precio_unitario,
               d.subtotal, d.descuento, d.total_venta
        FROM Detalle_venta d
        JOIN Productos p ON p.id_producto = d.id_producto
        WHERE d.id_venta = p_id_venta
        ORDER BY d.id_detalle
    ) LOOP
        v_hay_detalle := TRUE;
        DBMS_OUTPUT.PUT_LINE(r.nombre || ' x ' || r.cantidad || ' a $' || r.precio_unitario);
        DBMS_OUTPUT.PUT_LINE('Subtotal $' || r.subtotal || ' Descuento $' || r.descuento || ' Total $' ||
            r.total_venta);
    END LOOP;
    IF NOT v_hay_detalle THEN
        RAISE_APPLICATION_ERROR(-20006, 'La venta no tiene productos asociados');
    END IF;
    DBMS_OUTPUT.PUT_LINE('TOTAL BOLETA: $' || v_total);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('La venta ' || p_id_venta || ' no existe');
END MostrarBoleta;
/

CREATE OR REPLACE FUNCTION TotalVendidoHoy RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT NVL(SUM(total_venta), 0) INTO v_total
    FROM Venta
    WHERE fecha_venta >= TRUNC(SYSDATE)
      AND fecha_venta < TRUNC(SYSDATE) + 1;
    RETURN v_total;
END TotalVendidoHoy;
/

CREATE OR REPLACE PROCEDURE ResumenHoy IS
    CURSOR c_ventas IS
        SELECT id_venta, total_venta
        FROM Venta
        WHERE fecha_venta >= TRUNC(SYSDATE)
          AND fecha_venta < TRUNC(SYSDATE) + 1
```

Anexo A. Código completo (7/9)

```
        ORDER BY id_venta;
    v_cantidad NUMBER := 0;
    v_total NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE('----- RESUMEN DEL DIA ' || TO_CHAR(SYSDATE, 'DD-MM-YYYY') || ' -----');
    FOR r IN c_ventas LOOP
        v_cantidad := v_cantidad + 1;
        v_total := v_total + r.total_venta;
        DBMS_OUTPUT.PUT_LINE('Venta ' || r.id_venta || ': $' || r.total_venta);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Ventas de hoy: ' || v_cantidad);
    DBMS_OUTPUT.PUT_LINE('Total vendido: $' || v_total);
    DBMS_OUTPUT.PUT_LINE('Control: $' || TotalVendidoHoy);
END ResumenHoy;
/

CREATE OR REPLACE PROCEDURE ResumenEmpleado (
    p_id_empleado IN NUMBER
) IS
    v_nombre Empleado.nombre%TYPE;
    v_cantidad NUMBER;
    v_total NUMBER;
BEGIN
    SELECT nombre INTO v_nombre FROM Empleado WHERE id_empleado = p_id_empleado;
    SELECT COUNT(*), NVL(SUM(total_venta), 0)
    INTO v_cantidad, v_total
    FROM Venta
    WHERE id_empleado = p_id_empleado
      AND fecha_venta >= TRUNC(SYSDATE)
      AND fecha_venta < TRUNC(SYSDATE) + 1;
    DBMS_OUTPUT.PUT_LINE('Empleado: ' || v_nombre);
    DBMS_OUTPUT.PUT_LINE('Ventas de hoy: ' || v_cantidad);
    DBMS_OUTPUT.PUT_LINE('Total vendido: $' || v_total);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('El empleado no existe');
END ResumenEmpleado;
/

-- Pruebas
VARIABLE b_venta1 NUMBER
VARIABLE b_venta2 NUMBER

-- Probar funciones en consultas SQL
SELECT n AS cantidad,
       CalcularSubtotal(2, n) AS subtotal,
       CalcularDescuento(CalcularSubtotal(2, n)) AS descuento
FROM (SELECT LEVEL AS n FROM DUAL CONNECT BY LEVEL <= 3);

-- Consultar resumen sin ventas nuevas
EXEC ResumenHoy
SELECT TotalVendidoHoy AS total_hoy FROM DUAL;

SAVEPOINT antes_pruebas;
```

Anexo A. Código completo (8/9)

```
-- Registrar ventas de dos empleados
DECLARE
    v_stock_antes NUMBER;
    v_stock_despues NUMBER;
BEGIN
    SELECT stock INTO v_stock_antes FROM Productos WHERE id_producto = 2;
    PkgVentas.RegistrarVenta(1, 2, 2, :b_venta1);
    PkgVentas.RegistrarVenta(2, 2, 1, :b_venta2);
    SELECT stock INTO v_stock_despues FROM Productos WHERE id_producto = 2;
    DBMS_OUTPUT.PUT_LINE('Stock producto 2: ' || v_stock_antes || ' -> ' || v_stock_despues);
END;
/

-- Mostrar boletas y resumenes
EXEC MostrarBoleta(:b_venta1)
EXEC MostrarBoleta(:b_venta2)
EXEC ResumenEmpleado(1)
EXEC ResumenEmpleado(2)
EXEC ResumenHoy

-- Comprobar ventas, empleados, sucursales, totales y stock
DECLARE
    v_correctos NUMBER;
    v_detalles NUMBER;
    v_stock NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_correctos
    FROM Venta
    WHERE (id_venta = :b_venta1 AND id_empleado = 1 AND id_tienda = 1 AND total_venta = 27000)
       OR (id_venta = :b_venta2 AND id_empleado = 2 AND id_tienda = 2 AND total_venta = 15000);
    SELECT COUNT(*) INTO v_detalles FROM Detalle_venta
    WHERE id_venta IN (:b_venta1, :b_venta2);
    SELECT stock INTO v_stock FROM Productos WHERE id_producto = 2;
    IF v_correctos <> 2 OR v_detalles <> 2 OR v_stock <> 27 OR TotalVendidoHoy <> 42000 THEN
        RAISE_APPLICATION_ERROR(-20007, 'Revisar los resultados de la prueba');
    END IF;
    DBMS_OUTPUT.PUT_LINE('Prueba correcta: ventas, empleados, sucursales, total y stock');
END;
/

-- Probar ventas invalidas
DECLARE
    v_id_venta NUMBER;
BEGIN
    PkgVentas.RegistrarVenta(99, 2, 1, v_id_venta);
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE <> -20005 THEN RAISE; END IF;
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/

DECLARE
```

Anexo A. Código completo (9/9)

```
    v_id_venta NUMBER;
BEGIN
    PkgVentas.RegistrarVenta(1, 4, 999, v_id_venta);
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE <> -20003 THEN RAISE; END IF;
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/

-- Comprobar bloqueo de venta y detalle
BEGIN
    BEGIN
        UPDATE Venta SET total_venta = 0;
    EXCEPTION
        WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE(SQLERRM);
    END;
    BEGIN
        DELETE FROM Detalle_venta;
    EXCEPTION
        WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE(SQLERRM);
    END;
END;
/

-- Probar auditoria de precio
UPDATE Productos SET valor_unitario = 8500 WHERE id_producto = 1;
SELECT id_producto, precio_anterior, precio_nuevo
FROM Auditoria_Precio;

ROLLBACK TO antes_pruebas;
```